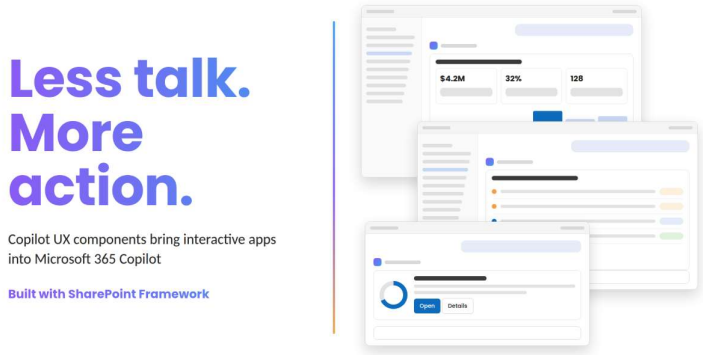


From chat to action: Copilot UX components and your SharePoint investments



Carlos Miguel Silva
CTO at BindTuning | Microsoft MVP | Speaker

October 29, 2026

Copilot has been good at answering. Now it can help people act.

For most organizations, Microsoft 365 Copilot has been a very capable reader. It summarizes documents, drafts emails and finds the policy you were looking for.

But the moment a user needs to *do* something - submit a request, approve a change, pick from a filtered list - Copilot hands them a link and sends them somewhere else. The conversation stops and the process starts in another app.

That gap is what **Copilot UX components** are designed to close, and it is why I think this is the most important change to the SharePoint Framework (SPFx) in years.

What they are, in plain terms

Copilot UX components are interactive pieces of UI - forms, grids, approval panels, charts, dashboards - that render directly inside the Microsoft 365 Copilot conversation.

They are built with SPFx, the same framework thousands of organisations already use for SharePoint web parts. A component is packaged together with a declarative agent, which is what makes it discoverable and callable from a Copilot chat.

The timeline so far:

- **June 2026** - announced as "SharePoint Copilot Apps".
- **July 2026** - public preview with SPFx 1.24.

- **September 2026** - renamed "UX components in Copilot", with general availability planned for October 2026.

In short: the user asks, Copilot responds with a working UI, and the user completes the task without leaving the conversation.

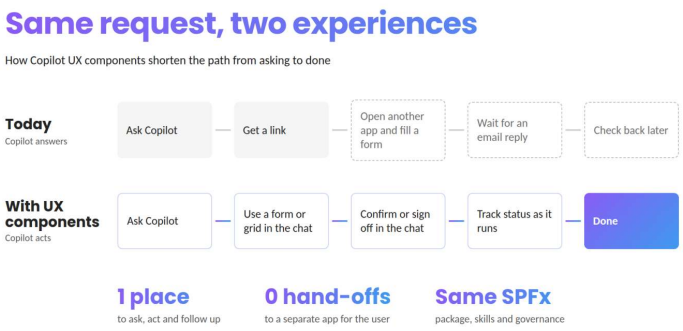
For business and communications leaders: why it matters

The real value is fewer context switches between asking and doing. A few scenarios that become realistic:

- **Requests and approvals.** An employee asks for a new project workspace and fills in a short guided form right in the chat. A manager approves it from an approval panel instead of hunting for an email.
- **Internal communications.** A communicator asks how last week's announcement performed and gets an interactive view, not a paragraph of numbers.
- **Employee services.** HR, IT or facilities requests start in the place people already ask questions, with the right fields and validation built in.

Microsoft groups the early scenarios into line-of-business agents, corporate communications and services agents, and management and governance agents.

There is also an investment argument. Existing SharePoint and SPFx work carries forward, so organizations extend what they already own rather than rebuilding it for a new surface.



For developers and IT: what's under the hood

If you have shipped an SPFx web part, most of this will feel familiar.

- **Same stack.** TypeScript and React (or Angular, Vue or Svelte), packaged as a standard .sppkg. One package can contain Copilot UX components, web parts and extensions side by side.
- **Same deployment model.** Components use the established SPFx deployment and hosting model, so they go through the tenant app catalog like everything else.
- **Same data access.** A component gets the same this.context clients as a web part, including spHttpClient for SharePoint REST and msGraphClientFactory for Microsoft Graph. SPFx handles the tokens.
- **Open foundation.** Components implement the MCP Apps model and are surfaced through a declarative agent you fully control.

- **Platform refresh.** SPFx 1.24 makes React 18 the default, adds Node.js 24 and 26 support, and moves the new SPFx CLI to GA in place of the Yeoman generator.

For ISVs and partners, the cross-tenant story is the big one: build once, deploy to many customer tenants. Microsoft has said store distribution starts at GA, initially through the SharePoint store.

A practical tip from building with it: load data in the view rather than in onInit(). onInit() blocks the first render, so the component shows nothing until it finishes.

Two things are still open: final GA licensing, and the cost of any APIs your component calls on every turn. Some services, such as Work IQ, are billed through Copilot Credits, which adds up at department scale.

The catch: governance becomes urgent

When Copilot can act, the quality of what sits behind it matters more. An interactive component inherits whatever permissions, ownership and sharing your tenant already has, good or bad.

Three things to check first:

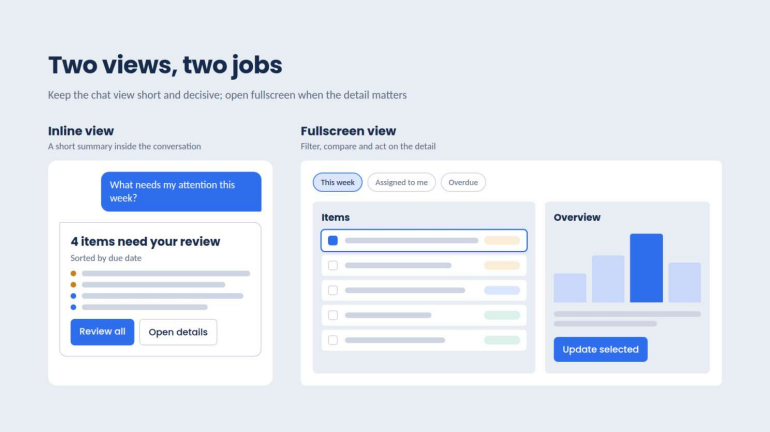
- **Oversharing** that went unnoticed in search becomes visible in a component.
- **Orphaned sites** with no clear owner become a source of wrong or stale answers.
- **API permissions** approved for one SPFx solution apply to every SPFx solution in the tenant, so app catalog hygiene matters.

My advice: treat your next permissions and oversharing review as part of your Copilot rollout, not a separate project.

What we're seeing in our own proofs of concept

We have been building internal proofs of concept on the preview, centred on a self-service request that needs governance and approval before anything is created. A few lessons so far:

- **Design two views, not one.** The inline view in the chat works best as a decision card: a recommendation, why it fits, and one or two clear actions. The detail - previews, forms, status - belongs in the fullscreen view.
- **Let the agent understand; let the component act.** Interpreting what the user means belongs in the declarative agent's instructions. The component receives structured input and renders. That boundary keeps both sides simpler to test.
- **Drive forms from a schema.** Different requests need different fields. Rendering them from a definition, rather than hard-coding each form, lets one component serve many scenarios.
- **Mock the backend behind clean interfaces.** In-memory services let us demo the full flow early, and make the real integration a swap rather than a rewrite.
- **Show the governance, not just the result.** People respond well when the UI explains *why* something needs approval and shows each provisioning step as it completes.
- **It's not an ACE.** Adaptive Card Extensions target Viva Connections and SharePoint pages. Copilot UX components are a different component type, and early briefs easily mix the two up.



Where to start

1. **Developers:** install the SPFx 1.24 tooling and build a first component in the Copilot Workbench. Pick one small, real process rather than a demo.
2. **IT and admins:** review app catalog ownership, existing SPFx packages and oversharing before components reach users.
3. **Business leaders:** list the two or three requests your teams make most often. Those are your first candidates.

Copilot has been the place where people ask. With UX components, it can become the place where work gets done - provided the foundations underneath are ready.

Sources

- Going beyond text in Microsoft 365 Copilot – Introducing SharePoint Copilot Apps - Microsoft 365 Developer Blog
- SPFx roadmap update – September 2026 - Microsoft 365 Developer Blog
- SPFx roadmap update – August 2026 - Microsoft 365 Developer Blog
- Overview of Copilot UX components - Microsoft Learn
- Access data from a Copilot UX component - Microsoft Learn
- Work IQ Answers sample - PnP



Carlos Miguel Silva

CTO at BindTuning | Microsoft MVP | Speaker

✓ Following

About
Accessibility
Talent Solutions
Professional Community Policies
Careers
Marketing Solutions
Privacy & Terms
Ad Choices
Advertising
Sales Solutions
Mobile
Small Business
Safety Center

Questions?
Visit our Help Center.

Manage your account and privacy
Go to your Settings.

Recommendation transparency
Learn more about Recommended Content.

Select Language
English (English)

LinkedIn Corporation © 2026